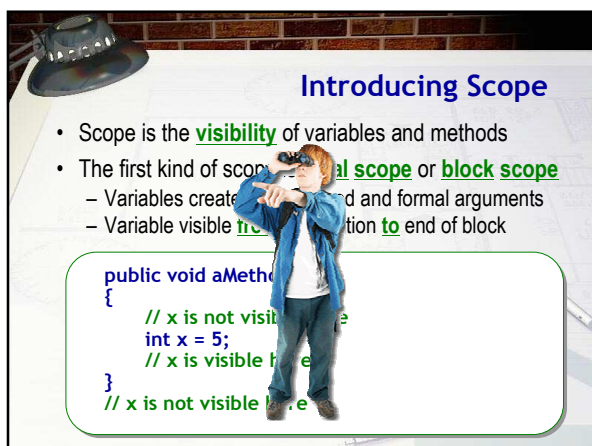
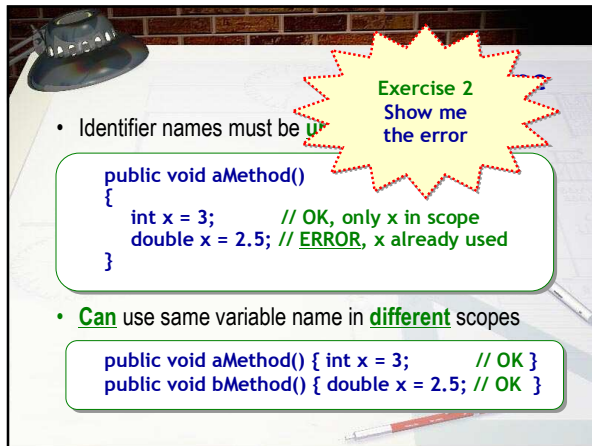










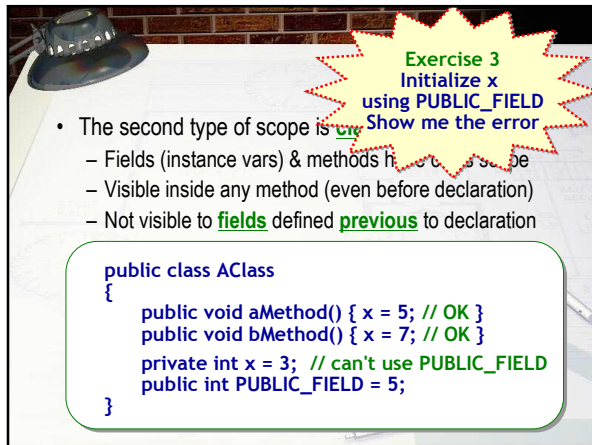
Exercise 2
Show me the error

- Identifier names must be unique

```
public void aMethod()
{
    int x = 3;           // OK, only x in scope
    double x = 2.5;      // ERROR, x already used
}
```

- Can use same variable name in **different** scopes

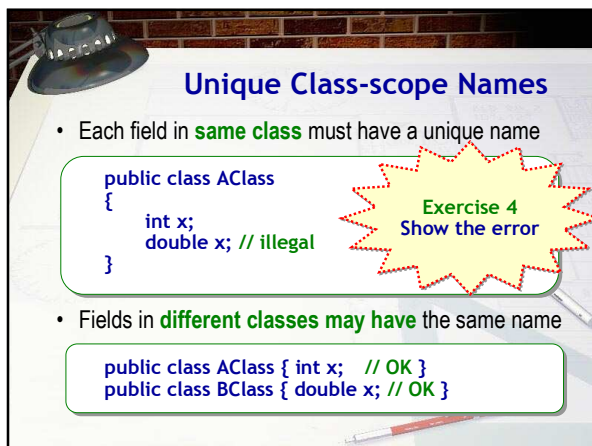
```
public void aMethod() { int x = 3;           // OK }
public void bMethod() { double x = 2.5;      // OK }
```



Exercise 3
Initialize x using PUBLIC_FIELD
Show me the error

- The second type of scope is **class**
 - Fields (instance vars) & methods have class scope
 - Visible inside any method (even before declaration)
 - Not visible to **fields** defined **previous** to declaration

```
public class AClass
{
    public void aMethod() { x = 5; // OK }
    public void bMethod() { x = 7; // OK }
    private int x = 3; // can't use PUBLIC_FIELD
    public int PUBLIC_FIELD = 5;
}
```



Unique Class-scope Names

- Each field in **same class** must have a unique name

```
public class AClass
{
    int x;
    double x; // illegal
}
```

Exercise 4
Show the error

- Fields in **different classes** may have the same name

```
public class AClass { int x; // OK }
public class BClass { double x; // OK }
```

Shadowing or Hiding

- A local variable **may** have the same name as a field
 - The local variable **hides** the field inside the local scope
 - You can access the **field** by using the keyword **this**

```
public class AClass
{
    double x = 2.2;
    public void aMethod()
    {
        int x;
        x = 7;
        this.x = 5.5;
    }
}
```

Exercise 5
Add a main()
Call aMethod()
View in Debugger

What Are "statics"

- Static fields:**
 - are shared among all objects in a class
 - act like "class wide" global variables
 - called "class variables" instead of "instance variables"
- Static methods:**
 - do not require an object to access them
 - are used by sending the message to the **class**
 - the **Math** class is composed of **static** methods
- "Effector" and "calculator methods are often **static**"

Defining a Class Variable

- Only difference **syntactically** is the modifier **static**
 - Doesn't matter which modifier comes first

```
public class XFighter extends SpaceCraft
{
    private int serialNo; // instance variable
    private static int numShips; // class variable
    public XFighter(int id) {
        serialNo = id;
        numShips++;
    }
    public void dieDieDIE() { numShips--; }
}
```

Defining static Methods

- To **define** a **static** method, use the **static** modifier
 - Only** access **static** fields or other **static** methods
 - You **cannot** access **any** instance variables

```
public static int getNumShips()
{
    // cannot call any instance methods in class
    // may call other static methods
    // may access static fields like numShips
    // cannot access serialNo from this method
    return numShips;
}
```

Calling static Methods

- Users **call** a **static** method by using the **class** name:

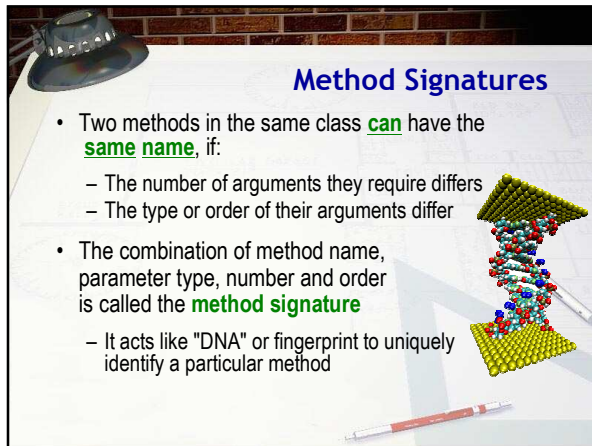
```
int ships = XFighter.getNumShips();
```

- No need to create an XFighter object first.
- In Java, users **may** call a **static** method using an instance of the **class** though

```
XFighter xf = new SFighter(52);
int ships = xf.getNumShips(); // very uncommon
```

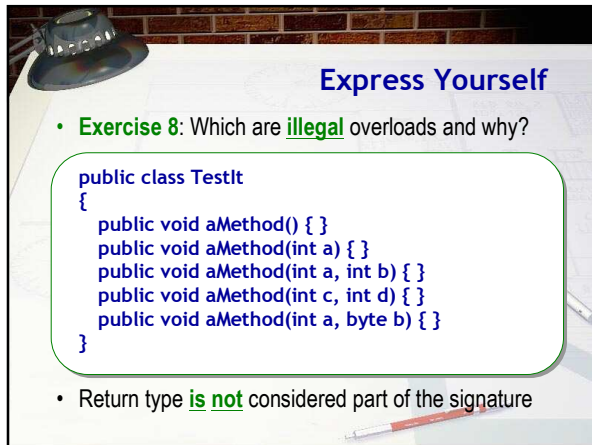
Express Yourself

- Exercise 6:** Using static variables and static methods
 - Import **Account.java** and **AccountTest.java**
 - Try to run **AccountTest**. Does it run? Why not?
 - Modify **Account** so that it works correctly
 - Compile, run and snap a picture with 5 accounts
- Exercise 7:** Add a **close()** method to **Account**
 - Set the balance to 0, append CLOSED to the account
 - Make sure you decrement the number of accounts
 - If an account number is odd, close it and print the number of accounts again at the end of the loop
 - Compile, run, and snap a picture with 5 accounts



Method Signatures

- Two methods in the same class **can** have the **same name**, if:
 - The number of arguments they require differs
 - The type or order of their arguments differ
- The combination of method name, parameter type, number and order is called the **method signature**
 - It acts like "DNA" or fingerprint to uniquely identify a particular method

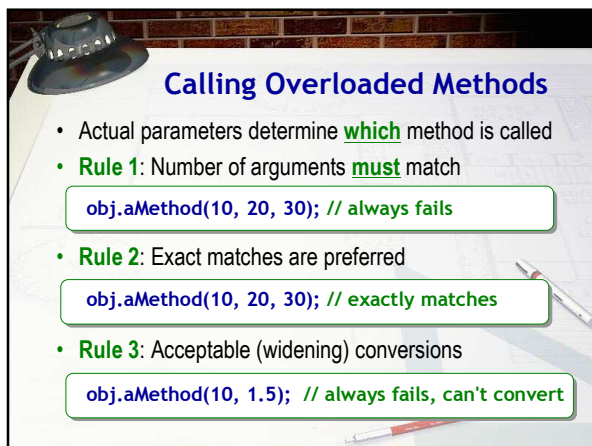


Express Yourself

- Exercise 8:** Which are **illegal** overloads and why?

```
public class TestIt
{
    public void aMethod() { }
    public void aMethod(int a) { }
    public void aMethod(int a, int b) { }
    public void aMethod(int c, int d) { }
    public void aMethod(int a, byte b) { }
}
```

- Return type **is not** considered part of the signature



Calling Overloaded Methods

- Actual parameters determine **which** method is called
- Rule 1:** Number of arguments **must** match
`obj.aMethod(10, 20, 30); // always fails`
- Rule 2:** Exact matches are preferred
`obj.aMethod(10, 20, 30); // exactly matches`
- Rule 3:** Acceptable (widening) conversions
`obj.aMethod(10, 1.5); // always fails, can't convert`

Good Arguments

- Here are a list of acceptable (widening) conversions

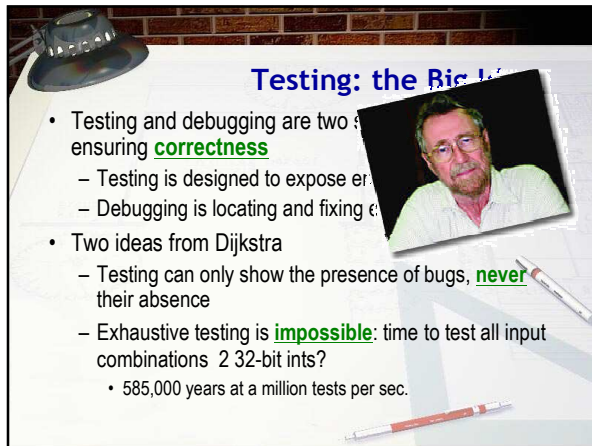
Formal Parameters	Acceptable	Unacceptable
byte	byte	everything else
short	byte, short	everything else
int	byte, short, int, char	everything else
long	byte, short, int, char, long	float, double
char	byte, short	everything else
long	byte, short, int, char, long	float, double
float	float, all integer types	double
double	all numeric types	none

Overloaded Methods

- Why bother?** More convenient coding
 - Imagine writing `System.out.println()`, etc.
- Exercise 9:** Overload Account ctors and methods:
 - Account (double initBal, String owner)** – initialize balance and owner; randomly generate account no.
 - Account (String owner)** – initialize owner; set initial balance to 0, randomly generate the account number.
 - Overload the **withdraw()** method with one that also takes a fee and deducts that fee from the account.
- Run `TestAccount2.java` to test your modified Account class. Snap a picture and paste it into `ic19.doc`

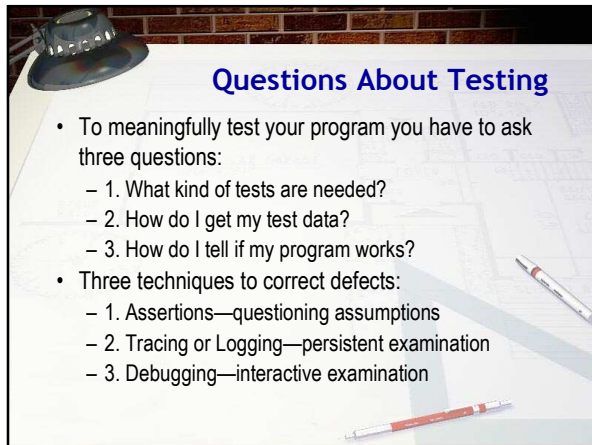
A Little More Eclipse

- Introducing **JUnit4** for writing unit tests
 - Using Eclipse to generate JUnit test cases
 - Using JUnit **annotations** to create test methods
 - Using **static** imports to import **Assert** class methods
 - Using different JUnit assertions
 - Writing a test fixture using the **setup()** method



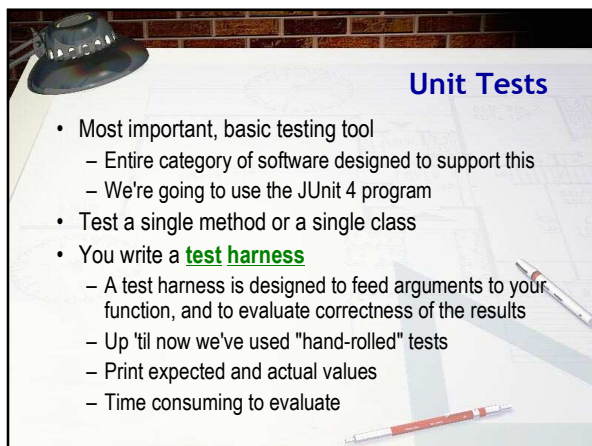
Testing: the Big M

- Testing and debugging are two sides of the same coin, ensuring **correctness**
 - Testing is designed to expose errors
 - Debugging is locating and fixing errors
- Two ideas from Dijkstra
 - Testing can only show the presence of bugs, **never** their absence
 - Exhaustive testing is **impossible**: time to test all input combinations 2 32-bit ints?
 - 585,000 years at a million tests per sec.



Questions About Testing

- To meaningfully test your program you have to ask three questions:
 - 1. What kind of tests are needed?
 - 2. How do I get my test data?
 - 3. How do I tell if my program works?
- Three techniques to correct defects:
 - 1. Assertions—questioning assumptions
 - 2. Tracing or Logging—persistent examination
 - 3. Debugging—interactive examination



Unit Tests

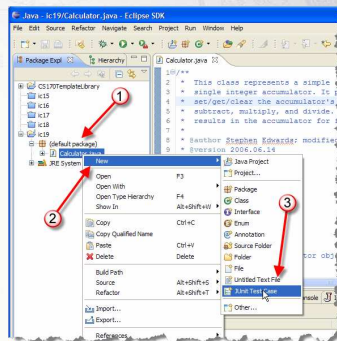
- Most important, basic testing tool
 - Entire category of software designed to support this
 - We're going to use the JUnit 4 program
- Test a single method or a single class
- You write a **test harness**
 - A test harness is designed to feed arguments to your function, and to evaluate correctness of the results
 - Up 'til now we've used "hand-rolled" tests
 - Print expected and actual values
 - Time consuming to evaluate

The Calculator Program

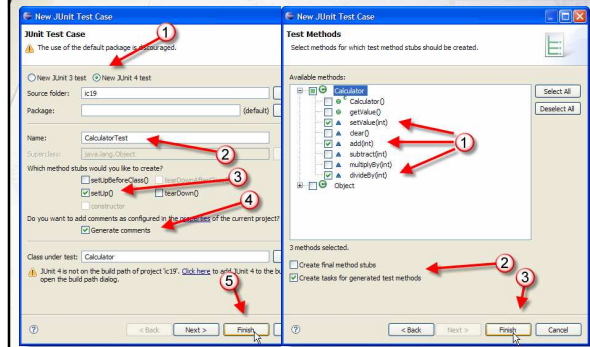
- **Exercise 10:** We're going to start by adding some tests to a program called **Calculator.java**
 - Import into your project from Q: drive
 - Open in editor and inspect
 - Active Checkstyle, set to CS 170 configuration
 - Fix all errors and shoot a screen-shot
- **Calculator.java**
 - A simple calculator with an accumulator
 - Methods to get/set/clear accumulator's value
 - Methods to add, subtract, multiply and divide

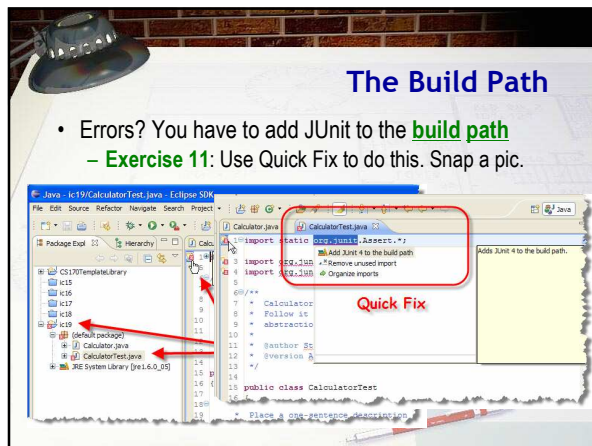
The Unit Test

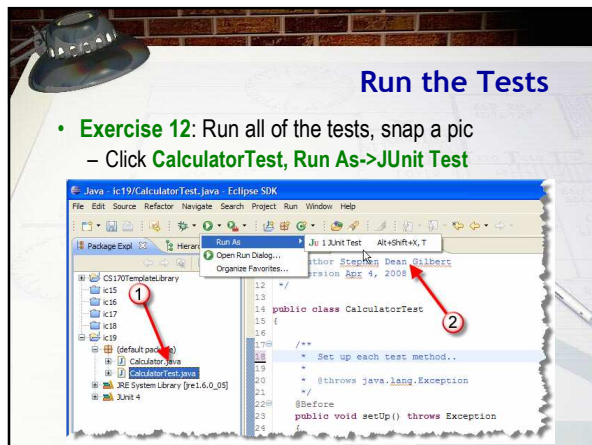
- A Test Case class will include a number of tests
 - Right-click Calculator.java
 - Choose New->JUnit Test Case

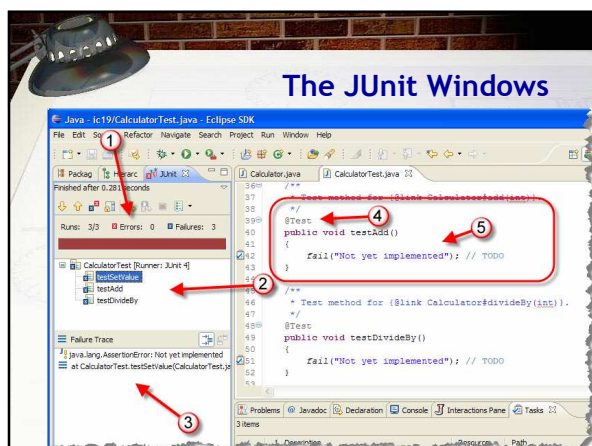


Test Class Wizard



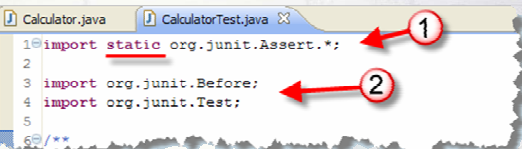






The JUnit4 Code

- **Static imports**: use **static** methods w/o class name

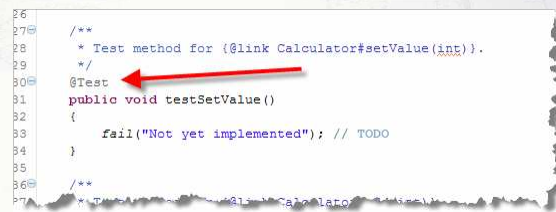


```
1 import static org.junit.Assert.*;
2
3 import org.junit.Before;
4 import org.junit.Test;
5
6 /**
```

- Also uses regular imports for each **annotation**
 - Annotations: special marker to identify code features
 - JUnit uses **@Test**, **@Before** and **@After**

Test Methods

- Any **public void** method taking no parameters
 - Must be preceded by **@Test** annotation
 - JUnit 3 required name to start with word **test**
 - Not required in JUnit 4, but common convention

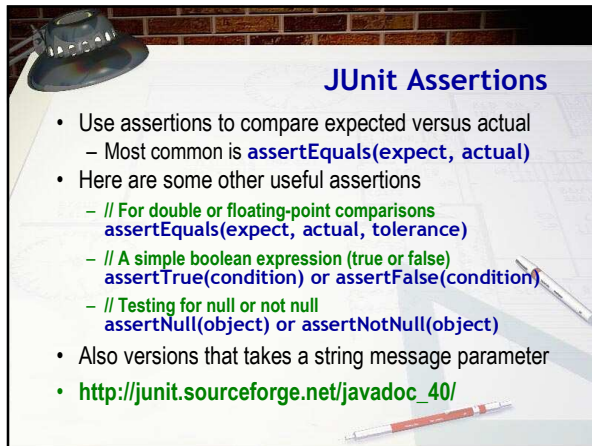


```
26
27 /**
28  * Test method for {@link Calculator#setValue(int)}.
29  */
30 @Test
31 public void testSetValue()
32 {
33     fail("Not yet implemented"); // TODO
34 }
35
36 /**
37  * Test method for {@link Calculator#setValue(int)}.
```

Writing Test Methods

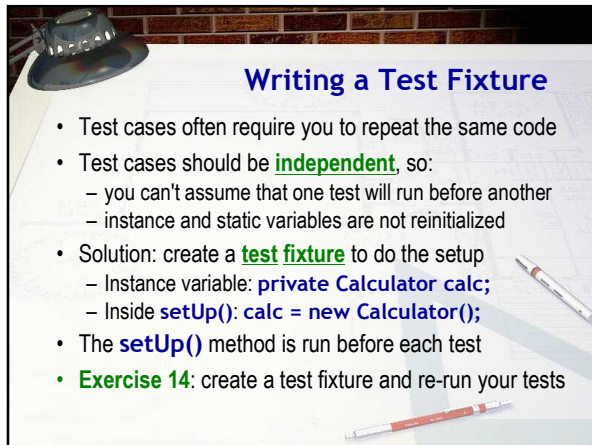
- Write a **separate** test method for each operation
 - Create an object in a known state
 - Call the method you are testing
 - Compare the **expected** state with the **actual** state
- **Exercise 13**: Add this code to the **testSetValue()** method, run your tests and shoot a screen-shot.

```
Calculator calc = new Calculator();
calc.setValue(5);
assertEquals(5, calc.getValue());
```



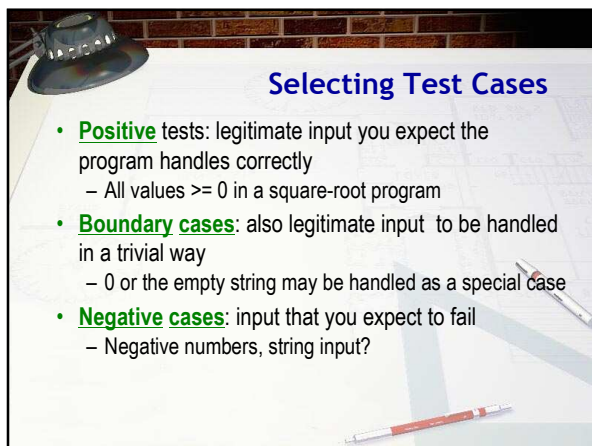
JUnit Assertions

- Use assertions to compare expected versus actual
 - Most common is `assertEquals(expect, actual)`
- Here are some other useful assertions
 - // For double or floating-point comparisons
`assertEquals(expect, actual, tolerance)`
 - // A simple boolean expression (true or false)
`assertTrue(condition)` or `assertFalse(condition)`
 - // Testing for null or not null
`assertNull(object)` or `assertNotNull(object)`
- Also versions that takes a string message parameter
- http://junit.sourceforge.net/javadoc_40/



Writing a Test Fixture

- Test cases often require you to repeat the same code
- Test cases should be **independent**, so:
 - you can't assume that one test will run before another
 - instance and static variables are not reinitialized
- Solution: create a **test fixture** to do the setup
 - Instance variable: `private Calculator calc;`
 - Inside `setUp()`: `calc = new Calculator();`
- The `setUp()` method is run before each test
- **Exercise 14:** create a test fixture and re-run your tests



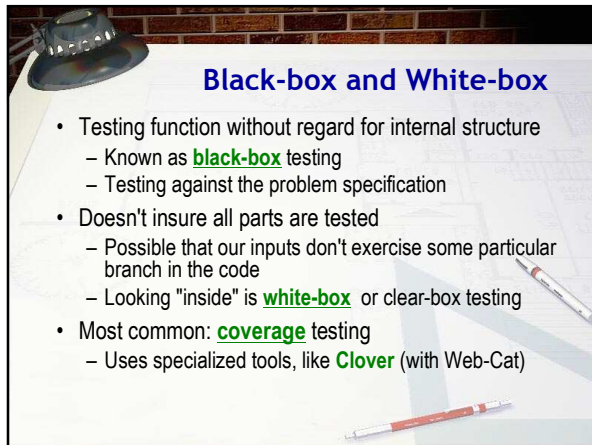
Selecting Test Cases

- **Positive** tests: legitimate input you expect the program handles correctly
 - All values ≥ 0 in a square-root program
- **Boundary cases:** also legitimate input to be handled in a trivial way
 - 0 or the empty string may be handled as a special case
- **Negative cases:** input that you expect to fail
 - Negative numbers, string input?



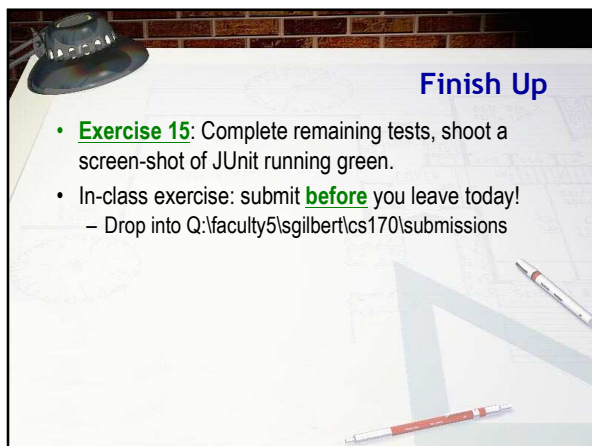
Regression Testing

- Whenever you fix a bug, you need to make sure it doesn't reappear later
 - Fixing a bug, only to have it reappear after further program modifications is called **cycling**
- Every time you fix a bug, you should **add a test** for it
 - Keep all tests; re-run every time you change program
- Testing your code against tests from past bugs is called **regression testing**
 - Another reason to keep your test data



Black-box and White-box

- Testing function without regard for internal structure
 - Known as **black-box** testing
 - Testing against the problem specification
- Doesn't insure all parts are tested
 - Possible that our inputs don't exercise some particular branch in the code
 - Looking "inside" is **white-box** or clear-box testing
- Most common: **coverage** testing
 - Uses specialized tools, like **Clover** (with Web-Cat)



Finish Up

- **Exercise 15:** Complete remaining tests, shoot a screen-shot of JUnit running green.
- In-class exercise: submit **before** you leave today!
 - Drop into Q:\faculty5\sgilbert\cs170\submissions
